Hair Cutting Robots

Lecture 3: Enums and Structs in Rust

Supervisor - Prof. Dr. Mohammed Aquil Mirza, COMP
Student Mentors - Mr. Ruan Haozhe, Mr. Jiang Suyu

May 2026
The Hong Kong Polytechnic University

1 Enum, Match, and impl

Why Enums?

What if `bool` didn't exist? How would we represent two-of-something?

First attempt: use integers
We could use `0` and `1`. But nothing stops someone from writing `2` or `-1`, and the meaning is not in the type.

Solution: define a new type
A type that only ever has the values we name. In Rust, that is an `enum`.

Defining an Enum

Syntax

```
enum Name {
    Variant1,
    Variant2,
}
```

Our running example

```
enum Bool {
    True,
    False,
}
```

A value of type `Bool` can only be `Bool::True` or `Bool::False`. Nothing else compiles.

Creating Enum Values

```
let yes = Bool::True;
let no  = Bool::False;
```

Access a variant with `EnumName::Variant`.

Variants Can Carry Data

Enum variants are not limited to plain names. Each variant can hold its own data, like a tagged union in C.

```
enum Shape {
    Circle(f32),                    // tuple-style payload
    Rectangle { w: f32, h: f32 }, // struct-style payload
    Origin,                         // no payload
}
```

We will explore this in a later lecture (e.g. `RuntimeCommand`). For today we stick to plain variants.

Why We Need `match`

We have a `Bool::True`. How do we *do* something different for each variant?

Without `match` there is no clean way. We cannot even compare with == unless we ask the compiler to derive `PartialEq`.

`match` is the natural tool for inspecting an enum.

`match` Basics

```
match yes {
    Bool::True  => println!("it is true"),
    Bool::False => println!("it is false"),
}
```

One arm per variant. The => separates the pattern from the action.

Exhaustiveness

Forget a variant

```
match yes {
    Bool::True => println!("yes"),
    // forgot Bool::False
}
```

Compiler error

```
error[E0004]: non-exhaustive patterns:
              `Bool::False` not covered
```

The compiler forces you to handle every variant. Add a new variant later and every `match` over that type breaks until you handle it. The compiler is your safety net.

`match` is an Expression

```
let label = match yes {
    Bool::True  => "yes",
    Bool::False => "no",
};
println!("{}", label);
```

Every arm produces a value, and the whole `match` evaluates to one. You can assign it, return it, or pass it as an argument.

What is an `impl` Block?

A block that attaches functions to a type.

```
impl Bool {
    fn flipped() -> Bool {
        Bool::False
    }
}

let b = Bool::flipped();
```

The function is called as `Bool::flipped()`. It lives in the type's namespace.

`Self` and Constructors

```
impl Bool {
    fn new() -> Self {
        Bool::False
    }
}
```

Inside an `impl`, `Self` is shorthand for the type being implemented. `Bool::new()` is the conventional constructor name.

Method Receivers: `self`, `&self`, `&mut self`

A method's first parameter says how it relates to the value it is called on.

```
impl Bool {
    fn consume(self)         { /* takes ownership; value gone after */ }
    fn consume_mut(mut self) { /* takes ownership, may mutate it    */ }
    fn read(&self)           { /* borrows immutably; default choice */ }
    fn modify(&mut self)     { /* borrows mutably; caller needs mut */ }
}
```

Rule of thumb

Pick the weakest one that does the job: `&self` first, then `&mut self`, then `self`.

Coming later

You will also meet `self: Box<Self>` and `self: Arc<Self>`, where the value is wrapped in a smart pointer. We cover them with ownership and shared state.

Methods with `&self`

```
impl Bool {
    fn is_true(&self) -> bool {
        match self {
            Bool::True  => true,
            Bool::False => false,
        }
    }
}

let b = Bool::True;
println!("{}", b.is_true());   // true
```

The first parameter `&self` makes the function callable with dot syntax: `b.is_true()`.

Method: `not`

```
impl Bool {
    fn not(&self) -> Bool {
        match self {
            Bool::True  => Bool::False,
            Bool::False => Bool::True,
        }
    }
}

let yes = Bool::True;
let no  = yes.not();
```

Method: and

```
impl Bool {
    fn and(&self, other: &Bool) -> Bool {
        match (self, other) {
            (Bool::True, Bool::True) => Bool::True,
            _ => Bool::False,
        }
    }
}
```

`match` on a tuple lets us inspect two values at once. `_` is the wildcard pattern: it matches anything.

Method: `or`

```
impl Bool {
    fn or(&self, other: &Bool) -> Bool {
        match (self, other) {
            (Bool::False, Bool::False) => Bool::False,
            _ => Bool::True,
        }
    }
}
```

Mirror image of `and`: only false-and-false yields false; everything else yields true.

One-line Derive Aside

```
#[derive(Debug, Clone, Copy)]
enum Bool {
    True,
    False,
}

println!("{:?}", Bool::True);   // True
```

`#[derive(...)]` asks the compiler to auto-generate common behavior. `Debug` enables `{:?}` printing; `Copy` lets us pass `Bool` by value without moves.

We will not go deeper today. The full traits lecture comes later.

Putting It Together (Definitions)

```
#[derive(Debug, Clone, Copy)]
enum Bool { True, False }

impl Bool {
    fn not(&self) -> Bool {
        match self {
            Bool::True  => Bool::False,
            Bool::False => Bool::True,
        }
    }
    fn and(&self, other: &Bool) -> Bool {
        match (self, other) {
            (Bool::True, Bool::True) => Bool::True,
            _ => Bool::False,
        }
    }
}
```

Putting It Together (`main`)

```
fn main() {
    let a = Bool::True;
    let b = Bool::False;
    println!("a       = {:?}", a);
    println!("not a   = {:?}", a.not());
    println!("a AND b = {:?}", a.and(&b));
}
```

Builds two `Bool` values, prints them, and exercises the methods defined on the previous slide.

In-Class Quiz: `xor`

Which of the following correctly implements `xor` as a method on `Bool`?

(1)

```
match (self, other) {
    (Bool::True,  Bool::False) => Bool::True,
    (Bool::False, Bool::True)  => Bool::True,
    _ => Bool::False,
}
```

(2) `self.and(other).not()`

(3)

```
match (self, other) {
    (Bool::True,  Bool::True)  => Bool::True,
    (Bool::False, Bool::False) => Bool::True,
    _ => Bool::False,
}
```

(4) `self.and(&other.not())`

In-Class Quiz: Answer

Answer: 1

Why the others fail

- **(2)** is NAND. It returns true unless both inputs are true.
- **(3)** is XNOR (equality). It returns true when the two inputs agree.
- **(4)** catches only the case `(True, False)`, missing the symmetric case `(False, True)`.

An equivalent definition using existing methods:

```
self.and(&other.not()).or(&other.and(&self.not()))
```

Reference Code

All runnable lecture code lives in one crate in the lecture repo:

```
Code_Week3/
  Cargo.toml
  src/bin/
    bool.rs
    vec2.rs
    rectangle.rs
```

Run a specific binary

```
cd Code_Week3
cargo run --bin bool       # or vec2, rectangle
```

Compare your own `xor` and `touches` against the reference bodies.

2 Structs

Why Structs?

A point in 2D has both an `x` and a `y`. Two separate variables fall apart fast. You have to keep them in sync by convention.

A **struct** bundles related fields into one named type. The compiler then treats them as one value.

Defining a Struct

```
struct Point {
    x: f32,
    y: f32,
}
```

Field name, then **type**. Trailing comma allowed and recommended.

Creating an Instance

```
let p = Point { x: 1.0, y: 2.0 };
println!("({}, {})", p.x, p.y);
```

Curly braces with `field: value`. Access fields with `.`.

Field Shorthand

```
let x = 1.0;
let y = 2.0;
let p = Point { x, y };   // same as { x: x, y: y }
```

When the variable name matches the field name, you can omit the value.

Mutable Structs

```
let mut p = Point { x: 1.0, y: 2.0 };
p.x = 5.0;
```

Rule
The whole instance is mutable or it is not. Rust has no per-field mutability.

Tuple Structs

A struct with positional fields and no names.

```
struct Rgb(u8, u8, u8);

let red = Rgb(255, 0, 0);
println!("{}", red.0);   // 255
```

Useful when field names would just be noise, e.g. wrapping a single value, or a small fixed tuple.

Unit Structs

A struct with no fields at all.

```
struct AlwaysEqual;

let x = AlwaysEqual;
```

Mostly a marker type. It comes in handy when we attach traits to it later.

Modules and Privacy

```
mod geometry {
    pub struct Point {
        pub x: f32,
        pub y: f32,
    }
}

fn main() {
    let p = geometry::Point { x: 1.0, y: 2.0 };
}
```

By default everything is private to its module. `pub` exposes a name to the outside.

Two Layers of `pub`

- `pub struct Point`: the **type** is visible.
- `pub x: f32`: the **field** is visible.

A `pub struct` with **private** fields cannot be constructed outside its module:

```
mod geometry {
    pub struct Point {
        x: f32,   // private!
        y: f32,
    }
}

fn main() {
    let p = geometry::Point { x: 1.0, y: 2.0 };
    // error: field ‘x‘ of struct ‘geometry::Point‘ is private
}
```

Why Hide Fields?

- Enforce invariants (e.g. "radius is always positive").
- Validate input at construction time.
- Free to change the internal layout later without breaking callers.

This forces us to provide a *constructor*, which is what `impl` gives us.

`impl` Block on a Struct

```
impl Point {
    fn new(x: f32, y: f32) -> Self {
        Point { x, y }
    }
}

let p = Point::new(1.0, 2.0);
```

Same syntax we saw on `Bool`. `Self` is `Point` inside the impl.

Constructor + Private Fields

```
mod geometry {
    pub struct Point {
        x: f32,   // private
        y: f32,
    }
    impl Point {
        pub fn new(x: f32, y: f32) -> Self {
            Point { x, y }
        }
        pub fn x(&self) -> f32 { self.x }
        pub fn y(&self) -> f32 { self.y }
    }
}

let p = geometry::Point::new(1.0, 2.0);
println!("{}", p.x());
```

Outside callers only see the public methods. The internal layout is ours to change.

Methods with `&self`

```
impl Point {
    pub fn length(&self) -> f32 {
        (self.x * self.x + self.y * self.y).sqrt()
    }
}

let p = Point::new(3.0, 4.0);
println!("{}", p.length());   // 5.0
```

Methods with `&mut self`

```
impl Point {
    pub fn scale(&mut self, k: f32) {
        self.x *= k;
        self.y *= k;
    }
}

let mut p = Point::new(1.0, 2.0);
p.scale(3.0);   // p is now (3.0, 6.0)
```

`&mut self` lets the method modify the instance. The caller must hold a `mut` binding.

Multiple `impl` Blocks

```
impl Point {
    pub fn new(x: f32, y: f32) -> Self {
        Point { x, y }
    }
}

impl Point {
    pub fn length(&self) -> f32 { /* ... */ }
}
```

Same type, split across blocks. Useful for grouping: constructors in one, math in another, I/O in a third.

Warm-up: `Vec2`

A 2D vector: two named fields plus a few methods.

```
#[derive(Clone, Copy, Debug)]
pub struct Vec2 {
    pub x: f32,
    pub y: f32,
}
impl Vec2 {
    pub fn new(x: f32, y: f32) -> Self {
        Self { x, y }
    }
    pub fn dot(self, other: Self) -> f32 {
        self.x * other.x + self.y * other.y
    }
    pub fn length(self) -> f32 {
        self.dot(self).sqrt()
    }
}
```

Why `self` (not `&self`) here?

`Vec2` is `Copy`: passing it by value is as cheap as a borrow, and the method signature reads more like math.

```
let a = Vec2::new(3.0, 4.0);
let b = Vec2::new(1.0, 2.0);

let d = a.dot(b);      // a and b are still usable
let l = a.length();    // a is still usable
```

For larger structs (not `Copy`) we go back to `&self` to avoid moves.

Vec3 in haircut_core

The simulator's real 3D vector: same shape as Vec2, one more axis.

```
#[derive(Clone, Copy, Debug, PartialEq, Default)]
pub struct Vec3 {
    pub x: f32,
    pub y: f32,
    pub z: f32,
}
impl Vec3 {
    pub const fn new(x: f32, y: f32, z: f32) -> Self {
        Self { x, y, z }
    }
    pub fn dot(self, other: Self) -> f32 {
        self.x * other.x + self.y * other.y + self.z * other.z
    }
}
```

Fields and methods are all pub; Vec3 is plain data.

In-Class Quiz: Do Two Rectangles Touch?

Given the struct

```
pub struct Rectangle {
    pub x: f32,        // left edge
    pub y: f32,        // bottom edge
    pub width:  f32,
    pub height: f32,
}
```

Common definition

Two axis-aligned rectangles *touch* if they share at least one point (edge contact counts).
Which of the following bodies correctly implements
`pub fn touches(&self, other:  &Rectangle) -> bool`?

In-Class Quiz: Options

(1)

```
self.x <= other.x + other.width
 && other.x <= self.x + self.width
 && self.y <= other.y + other.height
 && other.y <= self.y + self.height
```

(2)

```
self.x <= other.x + other.width
 && other.x <= self.x + self.width
```

(3)

```
self.x + self.width  < other.x
 || other.x + other.width  < self.x
 || self.y + self.height < other.y
 || other.y + other.height < self.y
```

(4)

```
self.x <= other.x + other.height
 && other.x <= self.x + self.height
 && self.y <= other.y + other.width
 && other.y <= self.y + self.width
```

In-Class Quiz: Answer

Answer: 1

Why the others fail

- **(2)** checks only the x-axis. Rectangles separated vertically are wrongly reported as touching.
- **(3)** is the *separation* condition: it returns `true` exactly when the rectangles do *not* touch (inverted result).
- **(4)** swaps `width` and `height` into the wrong axis tests. Compiles fine because both fields are `f32`.

In-Class Quiz: Answer (full method)

```
impl Rectangle {
    pub fn touches(&self, other: &Rectangle) -> bool {
        self.x <= other.x + other.width
            && other.x <= self.x + self.width
            && self.y <= other.y + other.height
            && other.y <= self.y + self.height
    }
}
```

3 Code Review: haircut_web::script

Where It Lives

`hcr-simulator/haircut_web/src/script.rs`

This is the only file students rewrite to design their own haircut. Every other module in the simulator stays the same.

What it exposes

- `MovementScript`: the contract the server calls each tick.
- `CutLine`: starting template (the script we already saw running).
- `RasterSweep`: reference implementation, selected with an env var.

The MovementScript Trait

```
pub trait MovementScript: Send {
    fn tick(&mut self, t: u64, sim: &mut Simulation);
}
```

A **trait** is a list of methods a type must provide. Full coverage in a later lecture.

How it is called

- The server calls `tick` about 30 times per second.
- `t`: the tick counter (0, 1, 2, ...).
- `sim`: the live simulation, taken as `&mut` so the script can send commands.
- `&mut self` lets the script remember state between calls.

CutLine: a Unit Struct

```
#[derive(Default)]
pub struct CutLine;

impl MovementScript for CutLine {
    fn tick(&mut self, t: u64, sim: &mut Simulation) {
        // ... script body ...
    }
}
```

Two new patterns

- `CutLine` has no fields. It is a *unit struct* (we previewed this earlier). It still needs to exist as a type so the trait impl can attach to it.
- `impl MovementScript for CutLine` reads as "`CutLine` fulfills the `MovementScript` trait".

Sending a Command

```
sim.apply_command(
    RuntimeCommand::Clipper(
        ClipperCommand::SetTargetXz { x, z }
    )
);
```

Two nested enums, both with payloads

- `RuntimeCommand::Clipper(...)` is a tuple-style variant. It wraps another enum.
- `ClipperCommand::SetTargetXz { x, z }` is a struct-style variant carrying named fields.
- Other variants: `MoveForward`, `ActivateCutting`, `Reset`, ...

This is the "variants can carry data" pattern we previewed in the enum chapter, used for real.

CutLine::tick — Three Phases

```
const APPROACH: u64 = 42;
if t < APPROACH {
    sim.apply_command(RuntimeCommand::Clipper(ClipperCommand::MoveForward));
    return;
}
if t == APPROACH {
    sim.apply_command(RuntimeCommand::Clipper(ClipperCommand::ActivateCutting));
}
const SWEEP_TICKS: f32 = 200.0;
let progress  = ((t - APPROACH) as f32 / SWEEP_TICKS).clamp(0.0, 1.0);
let x = -1.0 + 2.0 * progress * progress;
let z =  2.0 - 1.0 * progress;
sim.apply_command(RuntimeCommand::Clipper(ClipperCommand::SetTargetXz { x, z }));
```

Three phases driven entirely by `t`: **approach** (push forward), **activate** (turn on once), **sweep** (slide across). No mutable state on `CutLine` itself.